FORMATION « PYTHON EN 24 HEURES » | SÉANCE 09

Héritage, polymorphisme et méthodes spéciales

Chapitres 17 et 18 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Faire hériter une classe d'une autre et étendre `__init__` avec `super()`
- Redéfinir des méthodes et exploiter le polymorphisme
- Choisir entre héritage (EST UN) et composition (A UN)
- Implémenter `__repr__`, `__eq__`, `__add__`, `__len__` et `__getitem__`
- Comprendre l'ordre MRO en cas d'héritage multiple

Rappel de la séance 08

Vos classes savent naître, se protéger et se présenter : elles vont maintenant se transmettre des comportements et dialoguer avec les opérateurs de Python.

- `__init__` et self : la naissance et l'identité de l'instance
- Attributs d'instance contre attributs de classe partagés
- Encapsulation : `_interne`, `__privé`, `@property` avec validation
- `@classmethod` pour les fabriques, `@staticmethod` pour les utilitaires
- `__str__` : la première méthode spéciale rencontrée

L'héritage : réutiliser sans recopier

- class Voiture(Vehicule) : Voiture **hérite** de tout Vehicule
- Voiture est un Vehicule spécialisé : relation **EST UN**
- Les méthodes héritées fonctionnent sans être réécrites
- On peut **redéfinir** une méthode pour la spécialiser
- super() appelle la version de la classe parente

```
class Vehicule:
    def __init__(self, marque):
        self.marque = marque
    def klaxon(self):
        return "Tut tut"

class Voiture(Vehicule):
    def klaxon(self):
        return "Pouet pouet"

v = Voiture("Toyota")
print(v.marque, v.klaxon())
```

super().__init__() : étendre sans casser

```python
class Vehicule:
    def __init__(self, marque, roues):
        self.marque = marque
        self.roues = roues

class Moto(Vehicule):
    def __init__(self, marque, cylindree):
        super().__init__(marque, roues=2)
        self.cylindree = cylindree

m = Moto("Yamaha", 600)
print(m.marque, m.roues, m.cylindree)
```

super() délègue au parent la partie commune, puis Moto ajoute la sienne : zéro duplication du code d'initialisation.

Polymorphisme : même appel, comportements variés

- Une boucle peut appeler la même méthode sur des objets de classes différentes
- Chaque classe répond à sa façon : c'est le **polymorphisme**
- isinstance(x, Classe) teste la filiation en respectant l'héritage
- Le code appelant ne connaît que le contrat : ‘forme.aire()‘
- Ajouter une nouvelle forme ne modifie **aucune** ligne du code appelant

isinstance : tester la filiation

- ‘isinstance(x, Vehicule)‘ vaut True pour x Vehicule OU x Voiture
- L’héritage traverse le test : une fille EST UN parent
- Attention : trop de isinstance, c’est du code procédural déguisé
- Préférez le polymorphisme : appelez la méthode, la classe répond
- isinstance reste légitime aux frontières : validation, sérialisation

```
formes = [Cercle(1), Rectangle(2, 3)]

for forme in formes:
    print(isinstance(forme, Forme))

# le polymorphisme remplace les tests :
total = sum(f.aire() for f in formes)
```

Composition : la relation A UN

```
class Moteur:
    def __init__(self, puissance):
        self.puissance = puissance

class Voiture:                      # sans héritage !
    def __init__(self, marque, puissance):
        self.marque = marque
        self.moteur = Moteur(puissance)    # A UN moteur

v = Voiture("Toyota", 120)
print(v.moteur.puissance)
```

Une Voiture n'est pas un Moteur : elle en possède un. Héritage pour EST UN, composition pour A UN : la question à se poser avant chaque classe.

`__repr__` et `__str__` : deux vitrines

```python
class Duree:
    def __init__(self, heures, minutes):
        self.total = heures * 60 + minutes

    def __str__(self):
        return f"{self.total // 60} h {self.total % 60:02d}"

    def __repr__(self):
        return f"Duree({self.total // 60}, {self.total % 60})"

print(Duree(1, 90))        # 2 h 30
print([Duree(1, 90)])      # [Duree(2, 30)]
```

`__str__` pour l'utilisateur final (print), `__repr__` pour le développeur (console, listes, débogueur). Un `__repr__` honnête montre comment recréer l'objet.

`__eq__` et `__lt__` : comparer

```
class Duree:
    def __init__(self, heures, minutes=0):
        self.total = heures * 60 + minutes

    def __eq__(self, autre):
        return self.total == autre.total

    def __lt__(self, autre):
        return self.total < autre.total

print(Duree(1, 90) == Duree(2, 30))   # True
print(sorted([Duree(2, 0), Duree(1, 0)]))
```

Définir `__eq__` rend == correct ; définir `__lt__` suffit à sorted() pour trier les objets.

liste

```
class Playlist:
    def __init__(self, titres):
        self.titres = list(titres)

    def __len__(self):
        return len(self.titres)

    def __getitem__(self, index):
        return self.titres[index]

p = Playlist(["Awa", "Kofi"])
print(len(p))        # 2
print(p[0], p[-1])   # Awa Kofi
for titre in p:      # le for marche aussi !
    print(titre)
```

len() appelle __len__, les crochets appellent __getitem__ : une classe qui implémente ces deux méthodes devient iterable et indexable gratuitement.

Héritage multiple et MRO

- Une classe peut hériter de plusieurs parents : class C(A, B)
- Python résout un nom en remontant les parents dans un ordre précis
- Cet ordre est le **MRO** : Method Resolution Order
- C.__mro__ révèle la chaîne de recherche
- Les mixins offrent des comportements optionnels par héritage multiple léger

```
class Rechargeable:
    def recharger(self):
        return "batterie pleine"

class Roule:
    def rouler(self):
        return "je roule"

class Trottinette(Rechargeable, Roule):
    pass

t = Trottinette()
print(t.recharger(), t.rouler())
print(Trottinette.__mro__)
```

Attention

Le MRO et le diamant

En héritage multiple, deux parents peuvent définir la même méthode : c'est le premier de la liste `__mro__` qui gagne, pas le plus « logique ». Évitez les hiérarchies en diamant tant que le MRO n'est pas limpide pour vous, et préférez la composition quand l'héritage n'exprime pas clairement EST UN.

À retenir

L'essentiel de la séance 09

L'héritage exprime EST UN et se complète par super() ; la composition exprime A UN et reste souvent plus souple. Le polymorphisme fait parler chaque classe avec sa voix. Les méthodes spéciales (__repr__, __eq__, __lt__, __len__, __getitem__) branchent vos objets sur le langage lui même : print, ==, sorted, len et les crochets.

Travaux pratiques (1 h) : Héritage et méthodes spéciales

Cinq exercices pour brancher vos classes sur le langage. L'exercice 5 construit une petite bibliothèque de médias complète : gardez la, elle inspire le mini projet de la séance 10.

- **Exercice 1 (Basique)** : Vehicule et Voiture
- **Exercice 2 (Basique)** : __str__ et __repr__ sur Media
- **Exercice 3 (Intermédiaire)** : Formes polymorphes
- **Exercice 4 (Intermédiaire)** : Classe Duree avec __add__ et __eq__
- **Exercice 5 (Défi)** : Bibliothèque de médias indexable

Ouvrez le notebook « Seance_09_heritage_et_methodes_speciales.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 10 : le mini projet bancaire. Nous allons assembler trois classes qui se parlent, avec historique et validations. Relisez CompteSimple de la séance 08 : c'est son jumeau évolué qui ouvre le TP.